CP-SAT Solver

Page Summary

OR-Tools offers two main tools for solving integer programming problems:

- MPSolver (/optimization/lp/mpsolver), described in a previous section.
- The CP-SAT solver, which we describe next.

For an example that solves an integer programming problem using both the CP-SAT solver and the MPSolver wrapper, see Solving an Assignment Problem (/optimization/assignment/assignment_example).

Note: To increase computational speed, the CP-SAT solver works over the integers. This means you must define your optimization problem using integers only. If you begin with a problem that has constraints with non-integer terms, you need to first multiply those constraints by a sufficiently large integer so that all terms are integers. See Solving an Optimization Problem (/optimization/cp/integer_opt_cp) for an example.

The following sections present examples that show how to use the CP-SAT solver.

Example: finding a feasible solution

Let's start with a simple example problem in which there are:

- Three variables, x, y, and z, each of which can take on the values: 0, 1, or 2.
- One constraint: `x != y`

We'll start by showing how to use the CP-SAT solver to find a single feasible solution in all of the supported languages. While finding a feasible solution is trivial in this case, in more complex constraint programming problems it can be very difficult to determine whether there is a feasible solution.

For an example of finding an optimal solution to a CP problem, see Solving an Optimization Problem (/optimization/cp/cp_example).

Import the libraries

The following code imports the required library.

Python C++ (#c++) Java (#java) C# (#c) (#python)

```
from ortools.sat.python import cp_model
```

Declare the model

The following code declares the CP-SAT model.

Python C++ (#c++) Java (#java) C# (#c) (#python)

```
model = cp_model.CpModel()
```

Create the variables

The following code creates the variables for the problem.

Python C++ (#c++) Java (#java) C# (#c) (#python)

```
num_vals = 3
x = model.new_int_var(0, num_vals - 1, "x")
y = model.new_int_var(0, num_vals - 1, "y")
z = model.new_int_var(0, num_vals - 1, "z")
```

The solver creates three variables, x, y, and z, each of which can take on the values 0, 1, or 2.

Create the constraint

The following code creates the constraint `x != y`.

PythonC++ (#c++)Java (#java)C# (#c)
(#python)

```
model.add(x != y)
```

Call the solver

The following code calls the solver.

PythonC++ (#c++)Java (#java)C# (#c)
(#python)

```
solver = cp_model.CpSolver()
status = solver.solve(model)
```

CP-SAT return values

The CP-SAT solver returns one of the status values shown in the table below. In this example, the value returned is `OPTIMAL`.

Status	Description
`OPTIMAL`	An optimal feasible solution was found.
`FEASIBLE`	A feasible solution was found, but we don't know if it's optimal.
`INFEASIBLE`	The problem was proven infeasible.
`MODEL_INVALID`	The given CpModelProto didn't pass the validation step. You can get a detailed error by calling `ValidateCpModel(model_proto)`.
`UNKNOWN`	The status of the model is unknown because no solution was found (or the problem was not proven INFEASIBLE) before something caused the solver to stop, such as a time limit (/optimization/cp/cp_tasks#time-limit), a memory limit, or a custom limit set by the user.

These are defined in cp_model.proto (https://github.com/google/or-tools/blob/stable/ortools/sat/cp_model.proto).

Display the first solution

The following code displays the first feasible solution found by the solver. Note that the code checks whether the value of the `status` is `FEASIBLE` or `OPTIMAL`.

PythonC++ (#c++)Java (#java)C# (#c)
(#python)

```
  if status == cp_model.OPTIMAL or status == cp_model.FEASIBLE:
      print(f"x = {solver.value(x)}")
      print(f"y = {solver.value(y)}")
      print(f"z = {solver.value(z)}")
  else:
      print("No solution found.")
```

Run the program

The complete programs are shown the next section (#first_sol_program). When you run a program, it returns the first solution found by the solver:

```
x = 1
y = 0
z = 0
```

Complete programs

The complete programs are shown below.

PythonC++ (#c++)Java (#java)C# (#c)
(#python)

```
  """Simple solve."""
  from ortools.sat.python import cp_model
```

```
def simple_sat_program():
    """Minimal CP-SAT example to showcase calling the solver."""
    # Creates the model.
    model = cp_model.CpModel()

    # Creates the variables.
    num_vals = 3
    x = model.new_int_var(0, num_vals - 1, "x")
    y = model.new_int_var(0, num_vals - 1, "y")
    z = model.new_int_var(0, num_vals - 1, "z")

    # Creates the constraints.
    model.add(x != y)

    # Creates a solver and solves the model.
    solver = cp_model.CpSolver()
    status = solver.solve(model)

    if status == cp_model.OPTIMAL or status == cp_model.FEASIBLE:
        print(f"x = {solver.value(x)}")
        print(f"y = {solver.value(y)}")
        print(f"z = {solver.value(z)}")
    else:
        print("No solution found.")


simple_sat_program()
```

Finding all solutions

Next, we'll show how to modify the program above (#first_sol_program) to find all feasible solutions.

The main addition to the program is a **solution printer** a callback that you pass to the solver, which displays each solution as it is found.

Add the solution printer

The following code creates the solution printer.

PythonC++ (#c++)Java (#java)C# (#c)
(#python)

```
class VarArraySolutionPrinter(cp_model.CpSolverSolutionCallback):
    """Print intermediate solutions."""

    def __init__(self, variables: list[cp_model.IntVar]):
        cp_model.CpSolverSolutionCallback.__init__(self)
        self.__variables = variables
        self.__solution_count = 0

    def on_solution_callback(self) -> None:
        self.__solution_count += 1
        for v in self.__variables:
            print(f"{v}={self.value(v)}", end=" ")
        print()

    @property
    def solution_count(self) -> int:
        return self.__solution_count
```

Call the solver

The following code calls the solver, and passes the solution printer to it.

PythonC++ (#c++)Java (#java)C# (#c)
(#python)

```
solver = cp_model.CpSolver()
solution_printer = VarArraySolutionPrinter([x, y, z])
# Enumerate all solutions.
solver.parameters.enumerate_all_solutions = True
# Solve.
status = solver.solve(model, solution_printer)
```

Run the program

The complete program is shown in the next section (#all_solutions_program). When you run the program, it displays all 18 feasible solutions:

```
x=1 y=0 z=0
x=2 y=0 z=0
x=2 y=1 z=0
x=2 y=1 z=1
x=2 y=1 z=2
x=2 y=0 z=2
x=2 y=0 z=1
x=1 y=0 z=1
x=0 y=1 z=1
x=0 y=1 z=2
x=0 y=2 z=2
x=1 y=2 z=2
x=1 y=2 z=1
x=1 y=2 z=0
x=0 y=2 z=0
x=0 y=1 z=0
x=0 y=2 z=1
x=1 y=0 z=2
Status = FEASIBLE
```

Complete programs

The complete programs are shown below.

PythonC++ (#c++)Java (#java)C# (#c)
(#python)

```
from ortools.sat.python import cp_model


class VarArraySolutionPrinter(cp_model.CpSolverSolutionCallback):
    """Print intermediate solutions."""

    def __init__(self, variables: list[cp_model.IntVar]):
        cp_model.CpSolverSolutionCallback.__init__(self)
        self.__variables = variables
        self.__solution_count = 0
```

```
    def on_solution_callback(self) -> None:
        self.__solution_count += 1
        for v in self.__variables:
            print(f"{v}={self.value(v)}", end=" ")
        print()

    @property
    def solution_count(self) -> int:
        return self.__solution_count


def search_for_all_solutions_sample_sat():
    """Showcases calling the solver to search for all solutions."""
    # Creates the model.
    model = cp_model.CpModel()

    # Creates the variables.
    num_vals = 3
    x = model.new_int_var(0, num_vals - 1, "x")
    y = model.new_int_var(0, num_vals - 1, "y")
    z = model.new_int_var(0, num_vals - 1, "z")

    # Create the constraints.
    model.add(x != y)

    # Create a solver and solve.
    solver = cp_model.CpSolver()
    solution_printer = VarArraySolutionPrinter([x, y, z])
    # Enumerate all solutions.
    solver.parameters.enumerate_all_solutions = True
    # Solve.
    status = solver.solve(model, solution_printer)

    print(f"Status = {solver.status_name(status)}")
    print(f"Number of solutions found: {solution_printer.solution_count}")


search_for_all_solutions_sample_sat()
```

Except as otherwise noted, the content of this page is licensed under the Creative Commons Attribution 4.0 License (https://creativecommons.org/licenses/by/4.0/), and code samples are licensed under the Apache 2.0 License (https://www.apache.org/licenses/LICENSE-2.0). For details, see the Google Developers Site Policies

(https://developers.google.com/site-policies). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2024-08-28 UTC.